

Methodology: Program Graphs and Intermediate Representations

Program graph

Let $G = (V, E)$ denote a directed graph whose vertices V represent program entities and whose edges E represent relationships. COGANT assigns each node a **kind** (for example function, variable, type) and a **semantic role** (for example definition versus use). Each node and edge may carry:

- ▶ A **stable identifier** for persistence across runs when hashing allows.
- ▶ **Type strings** and attributes recovered from the front end.
- ▶ A **confidence** score in $[0, 1]$ and **provenance** metadata (source code, type system, control flow, heuristic, external tool).

This follows the same philosophical line as code property graphs that fuse AST, control flow, and data flow into one analyzable structure [yamaguchi2014modeling], but orients the representation toward **Generalized Notation Notation (GNN) export** and, optionally, tensorized views of the same graph: node kinds and roles map to discrete indices in both forms, and optional

Progressive IRs

Processing proceeds through a sequence of representations (see `../cogant/docs/SPEC.md`):

1. **Repo IR** — entities and relationships extracted from parsers.
2. **Program graph IR** — consolidated graph with deduplication and metadata.
3. **Semantic mapping IR** — output of the translation rule engine.
4. **State space IR** — variables, actions, transitions, observations.
5. **Process model IR** — higher-level control patterns where implemented.
6. **Validation IR** — coverage, confidence analysis, schema checks.

Not every stage is equally complete for every repository; the SPEC marks partial areas (translation rules, state space, Rust acceleration) explicitly.

Translation rules

The **translation** stage applies declarative rules that refine roles, attach labels, and adjust confidence. Concurrency targets and layering are described in `../cogant/docs/ARCHITECTURE.md`. The rule engine composes passes over the graph in a **fixpoint loop**: rules are re-applied until no new semantic mappings emerge or a configurable iteration cap is reached. Overlapping rules may require priority ordering as documented there; when multiple rules claim the same graph fragment, the engine retains the mapping with the highest confidence score, following the principle that edit representations should be composable [yin2019learning].

Fixpoint iteration, conflict resolution, and coverage

The shipped `TranslationEngine` in `../cogant/py/cogant/translate/engine.py` realizes the fixpoint loop concretely. Each iteration walks every registered rule once: the engine calls `rule.matches(graph, query)` to collect candidate fragments, then calls `rule.apply(graph, match)` on each, accumulating any resulting `SemanticMapping` objects keyed by their stable IDs. A per-pass counter tracks mappings that were

Confidence Scoring and Evidence Tiers

The shipped `ConfidenceModel` in

`../cogant/py/cogant/translate/confidence.py` computes a scalar score in $[0, 1]$ from:

- ▶ Average confidence over provenance records.
- ▶ An evidence-diversity term (scaled, capped).
- ▶ A parser certainty factor applied multiplicatively.
- ▶ Conflict penalties subtracted after scaling.

$$c = \max(0, \min(1, (\bar{e} + \delta_d) \cdot \kappa - \pi)) \quad (2)$$

Here $\bar{e}$ is the mean evidence confidence, δ_d is the diversity bonus (bounded), κ is parser certainty, and π aggregates conflict penalties. **Tiers** (for example static-only versus static-plus-runtime) are assigned from score thresholds and evidence source tags (`determine_confidence_tier`); see the same module for named thresholds and enum values. The manuscript does not duplicate those literals so they cannot drift from code.

State space and behavior

Where traces or coverage are available, **dynamic** extraction feeds the state-space compiler. The goal is a compact behavioral model: states, actions, transitions, and observations that sit alongside the static graph for tasks that require execution-sensitive features.

The shipped `StateSpaceCompiler` in

```
../cogant/py/cogant/statespace/compiler.py
```

 constructs this model in several coordinated passes driven by the semantic mappings and the underlying program graph.

State variables are identified by a `StateVariableExtractor` that traverses graph nodes carrying `READS` and `WRITES` edges: any node that participates in a write is a candidate hidden-state variable, and its type, cardinality, and initial confidence are derived from the node's recovered type string and the provenance of the rules that produced the associated mapping.

Actions are extracted from semantic mappings of kind `ACTION`, with parameters read from node metadata (typically the parser-recovered function signature) effects traced through

GNN export (Generalized Notation Notation)

Exports package the program graph and compiled state-space model into the Active Inference Institute's **Generalized Notation Notation** plus a small set of interop targets:

- ▶ **GNN Markdown** (`model.gnn.md`) — canonical, human-readable Generalized Notation Notation organized into the section order defined in `../cogant/docs/GNN_EXPORT.md` (Model Metadata, Repository Metadata, Source Coverage, State Space, Observation Modalities, Actions/Policies, Connections, Factors, Transition Structure, Likelihood Structure, Preferences/Constraints, Time Settings, Parameterization, Ontology Mapping, Provenance, Confidence Scores, Rendering Hints, Validation Notes).
- ▶ **GNN JSON** (`model.gnn.json`) — machine-readable equivalent of the markdown sections, plus companion JSON files per section (`state_space.json`, `observations.json`, `actions.json`, `transitions.json`, and so on).
- ▶ **Interop exports** — GraphML and Parquet for analysis in

Error handling philosophy

The implementation distinguishes fatal configuration or parse failures from per-file errors that allow partial results. Validation aggregates warnings and inconsistencies into a report that travels with the bundle. This mirrors the layered error categories documented in the architecture (fatal, error, warning, info).